Iker Acevedo Donate

CampusFP

Memoria TFG

1. De qué trata mi idea y por qué la propongo

La idea de este proyecto surge de un problema que casi todos los estudiantes y tutores hemos vivido: el caos administrativo que suponen las prácticas. Actualmente, la gestión está muy fragmentada. El alumno está en la empresa, el tutor en el centro y la comunicación entre ellos suele ser un lío de correos electrónicos que se pierden, llamadas o documentos de Excel que van y vienen. Esto hace que sea muy difícil llevar un control real de lo que el alumno hace día a día.

Mi aplicación busca centralizar todo esto en un único entorno digital. Aunque la plataforma está pensada para gestionar todo el ciclo (desde que una empresa publica una oferta hasta que se firma el convenio), **este TFG se va a centrar sobre todo en la parte del seguimiento y la comunicación**. Me parece la parte más importante porque es donde más fallos hay actualmente. Quiero que el tutor sepa de verdad cómo le va al alumno y que el alumno tenga un sitio fácil donde reportar cualquier problema sin que se quede en el olvido.

2. Objetivos del proyecto

- **Comunicación centralizada**: Crear un chat interno vinculado a la práctica para que las dudas y avisos no se mezclen con el correo personal.
- **Seguimiento real**: Que el alumno registre sus tareas y sus horas semanalmente de forma sencilla.
- **Gestión de incidencias**: Si pasa algo en la empresa, el alumno puede dar la alarma con un botón y el tutor recibe un aviso directo.
- **Supervisión del centro**: Que el centro educativo tenga una pantalla donde ver el estado de todos sus alumnos a la vez.

3. Análisis de Requisitos (Especificaciones Técnicas)

Para complementar los objetivos anteriores, he definido los requisitos técnicos que debe cumplir el sistema en esta primera fase:

3.1 Requisitos Funcionales (RF)

- **RF-01 (Gestión de Usuarios):** El sistema debe permitir el registro y login de usuarios con diferentes roles (Alumno, Tutor, Admin).
- **RF-02 (Perfil de Usuario):** El usuario debe poder visualizar sus datos de perfil una vez autenticado mediante token.
- **RF-03 (Gestión de Maestros):** Capacidad de listar Centros Educativos y Empresas colaboradoras para poblar los formularios del sistema.
- **RF-04 (Gestión de Prácticas):** El sistema permite la creación, consulta y actualización de convenios de prácticas por parte de administradores y tutores.
- **RF-05 (Seguridad por Roles):** Implementación de niveles de acceso estrictos; los alumnos solo acceden a su información y los administradores gestionan el sistema completo.
- **RF-06 (Seguimiento Diario):** El alumno puede registrar cada día qué ha hecho, cuántas horas ha trabajado y una descripción de las tareas. La aplicación lleva la cuenta automáticamente de las horas acumuladas frente al total del convenio.
- **RF-07 (Validación en Dos Pasos):** Los partes del alumno tienen que pasar primero por el tutor de empresa y después por el tutor del centro, igual que ocurre con los papeles actualmente. Si la empresa rechaza un parte, el sistema avisa automáticamente al tutor del centro.

3.2 Requisitos No Funcionales (RNF)

- **RNF-01 (Seguridad):** Uso de cifrado BCrypt para contraseñas y autenticación Stateless mediante JWT.
- **RNF-02 (Integridad):** Garantizar la integridad referencial de los datos mediante el motor PostgreSQL.

4. Arquitectura y Tecnologías

Para el desarrollo de este proyecto, he decidido evolucionar la idea inicial hacia una Arquitectura Cliente-Servidor (basada en API). Esta estructura separa completamente los datos del servidor de lo que el usuario ve en la pantalla.

- **Frontend (Aplicación Cliente):** Para la implementación del frontend se ha seguido una arquitectura de capas (Data, Domain, Presentation) sobre Flutter. Se utiliza Dio como cliente HTTP avanzado para la gestión de peticiones REST y Provider como motor de inyección de dependencias y gestión de estado. La seguridad se garantiza mediante interceptores que inyectan automáticamente el token JWT en las cabeceras de cada petición.

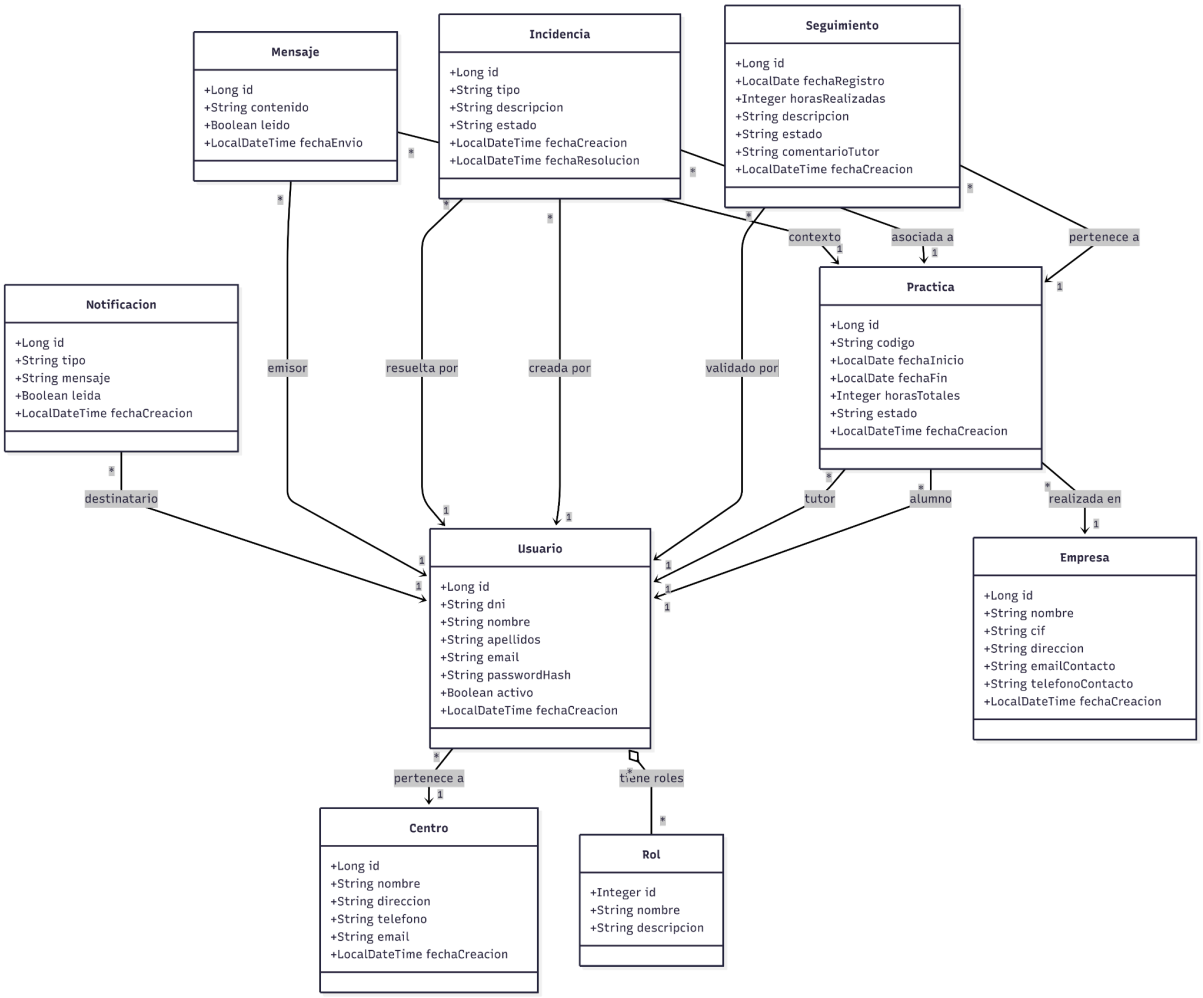
- **Backend (API REST):** He optado por **Spring Boot con Java 21**. He implementado una arquitectura de seguridad Stateless basada en **JWT (JSON Web Tokens)**, eliminando las sesiones en servidor para que el sistema sea más rápido y seguro.

- **Base de Datos:** He utilizado **PostgreSQL** (en lugar de MySQL) por su robustez en el manejo de relaciones complejas. Uso la herramienta **Flyway** para gestionar el historial de cambios de las tablas automáticamente.

4.1. Diseño de la Lógica (Backend)

Para organizar el código del servidor, he seguido un patrón orientado a objetos, separando claramente los Controladores (que reciben las peticiones de Flutter), los Servicios (donde está la lógica y las validaciones) y los Repositorios (que hablan con la base de datos).

A continuación, presento el diagrama de clases (UML) del núcleo del sistema, centrado en la autenticación y la gestión de entidades maestras (Centros y Empresas), que constituye el primer gran hito de desarrollo del proyecto.



En este segundo hito le he dado mucha más profundidad al backend. Lo más importante ha sido añadir una seguridad real basada en roles, no solo el login. Ahora cada pantalla de la API tiene una restricción concreta: un alumno con un token válido no puede crear prácticas

ni validar seguimientos aunque lo intente, porque el sistema lo bloquea antes de ejecutar nada. Para conseguir esto he usado la anotación `@PreAuthorize` de Spring Security directamente sobre cada método del controlador.

Para asegurarme de que todo esto funciona correctamente, he escrito 10 tests de integración. Estos tests no solo comprueban que las cosas buenas funcionan (que un tutor puede validar un seguimiento), sino también que las cosas malas se bloquean (que un alumno recibe un error 403 si intenta hacer algo que no le toca). Tener esta batería de tests me da confianza para seguir añadiendo funcionalidades sin romper lo que ya estaba.

4.2. El módulo de seguimientos: cuando el diseño inicial no reflejaba la realidad

Aquí quiero explicar algo que me parece interesante porque ilustra bien cómo evoluciona un proyecto real.

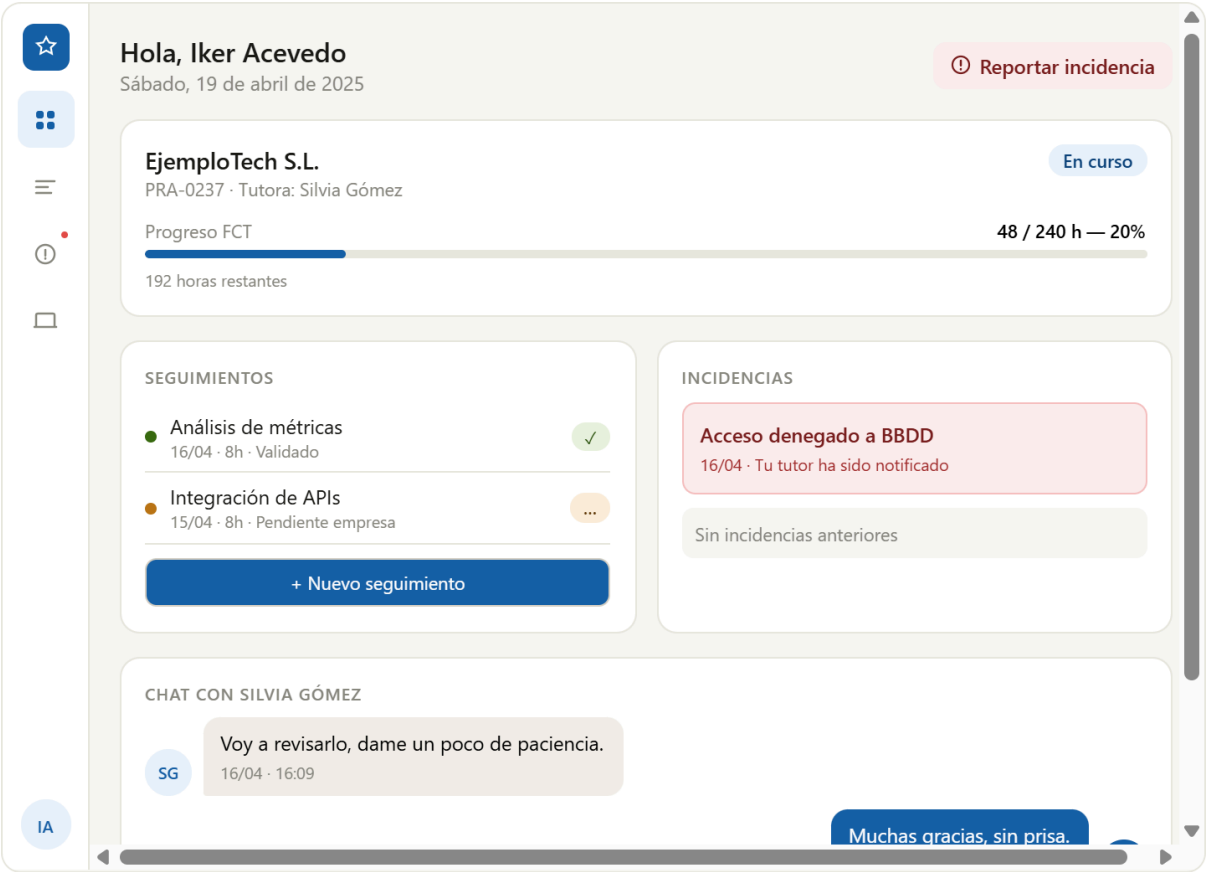
Cuando diseñé el módulo de seguimientos al principio, pensé en una validación simple: el alumno registra sus horas, un tutor las aprueba o rechaza, fin. Pero al ponerme a implementarlo en detalle me di cuenta de que ese diseño no reflejaba cómo funcionan las prácticas en la realidad.

El problema es que hay dos tutores con funciones completamente distintas. El tutor de empresa está ahí todos los días con el alumno y sabe exactamente lo que ha hecho esa semana. Él es quien firma el parte en papel actualmente. El tutor del centro, en cambio, no está en la empresa: su trabajo es supervisar que el proceso formativo va bien, que el alumno no tiene problemas, y que se están cumpliendo los objetivos del ciclo. Son cosas distintas y mezclarlas en una sola validación no tenía sentido.

La solución que adopté fue un flujo en dos pasos. Primero valida el tutor de empresa (¿son correctas las horas y las tareas?). Solo después de eso puede revisar el tutor del centro (¿va bien el alumno en general?). Para que esto funcione, el parte puede estar en cuatro estados distintos: esperando la firma de la empresa, esperando el visto bueno del centro, completado, o rechazado.

El caso del rechazo también lo pensé con cuidado. Si la empresa rechaza un parte, el sistema crea automáticamente una incidencia que el tutor del centro puede ver sin que el alumno tenga que hacer nada. Esto es importante porque un rechazo puede ser una tontería (el alumno puso mal las horas), pero también puede indicar que algo va mal con la empresa. En cualquier caso, el tutor del centro tiene que saberlo.

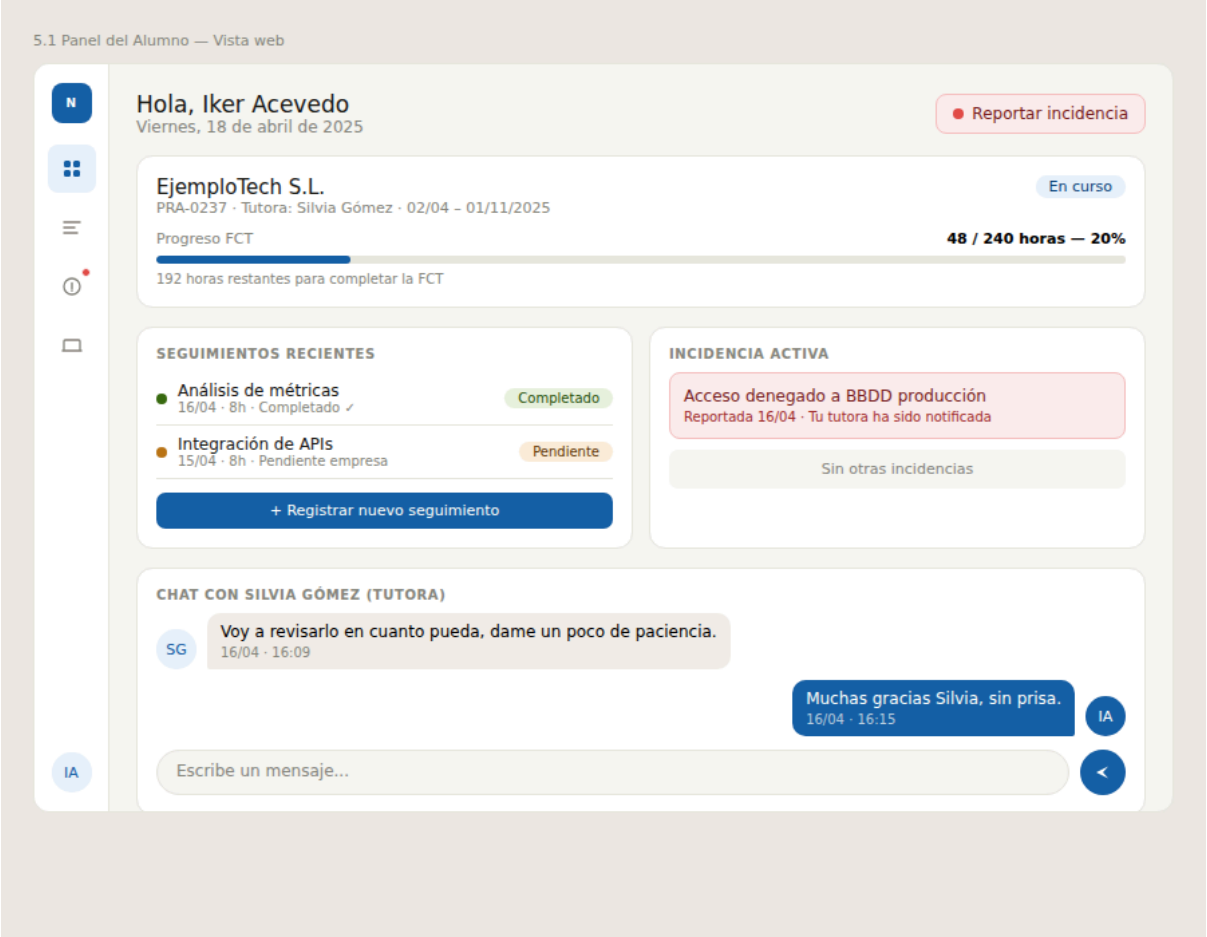
Detectar este problema antes de construir las pantallas fue una suerte, porque cambiarlo ahora tiene un coste bajo. Si lo hubiera descubierto después de tener el frontend hecho, habría tenido que rehacer todo.



5. Diseño de la Interfaz por Roles

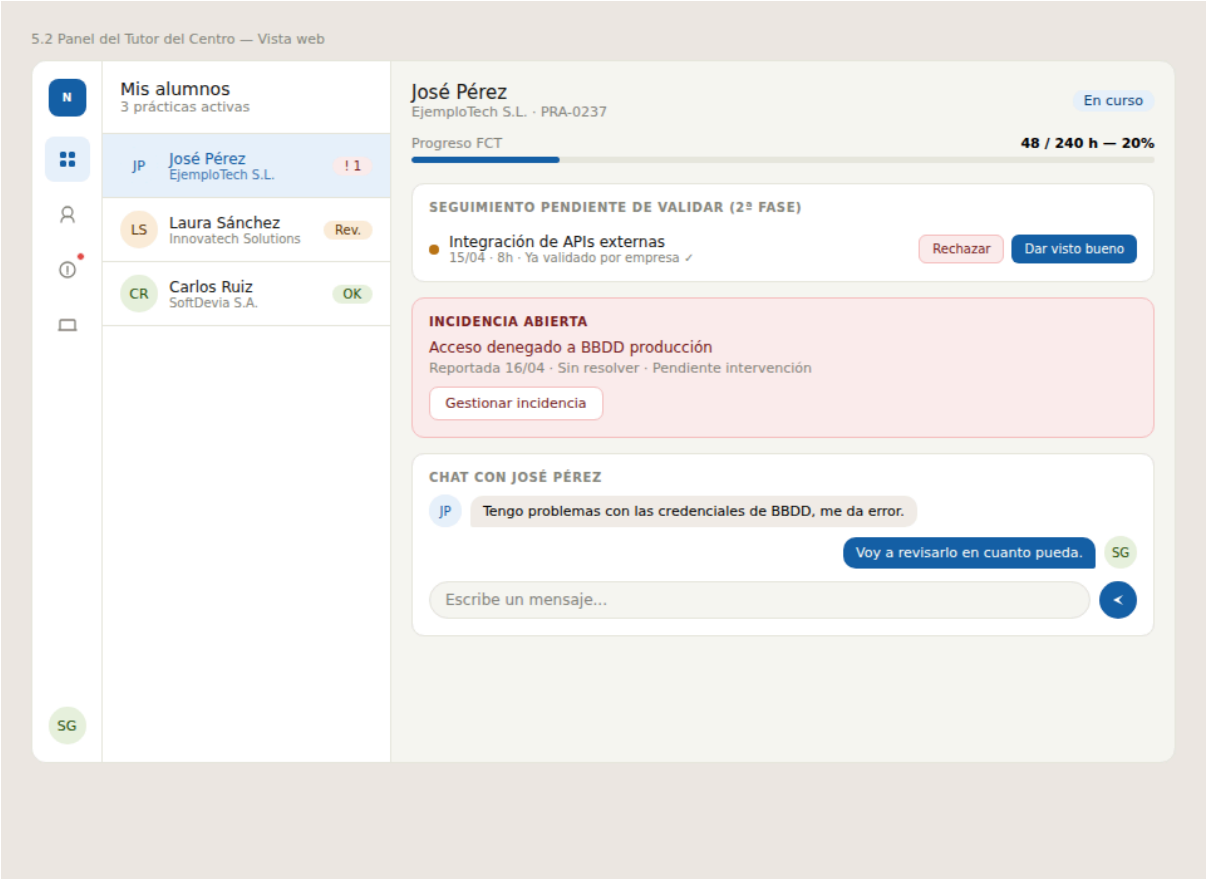
Aquí es donde se ve cómo funciona la aplicación. He diseñado cuatro paneles distintos porque cada usuario necesita cosas diferentes. Y luego he generado también la versión móvil.

5.1. Panel del Alumno



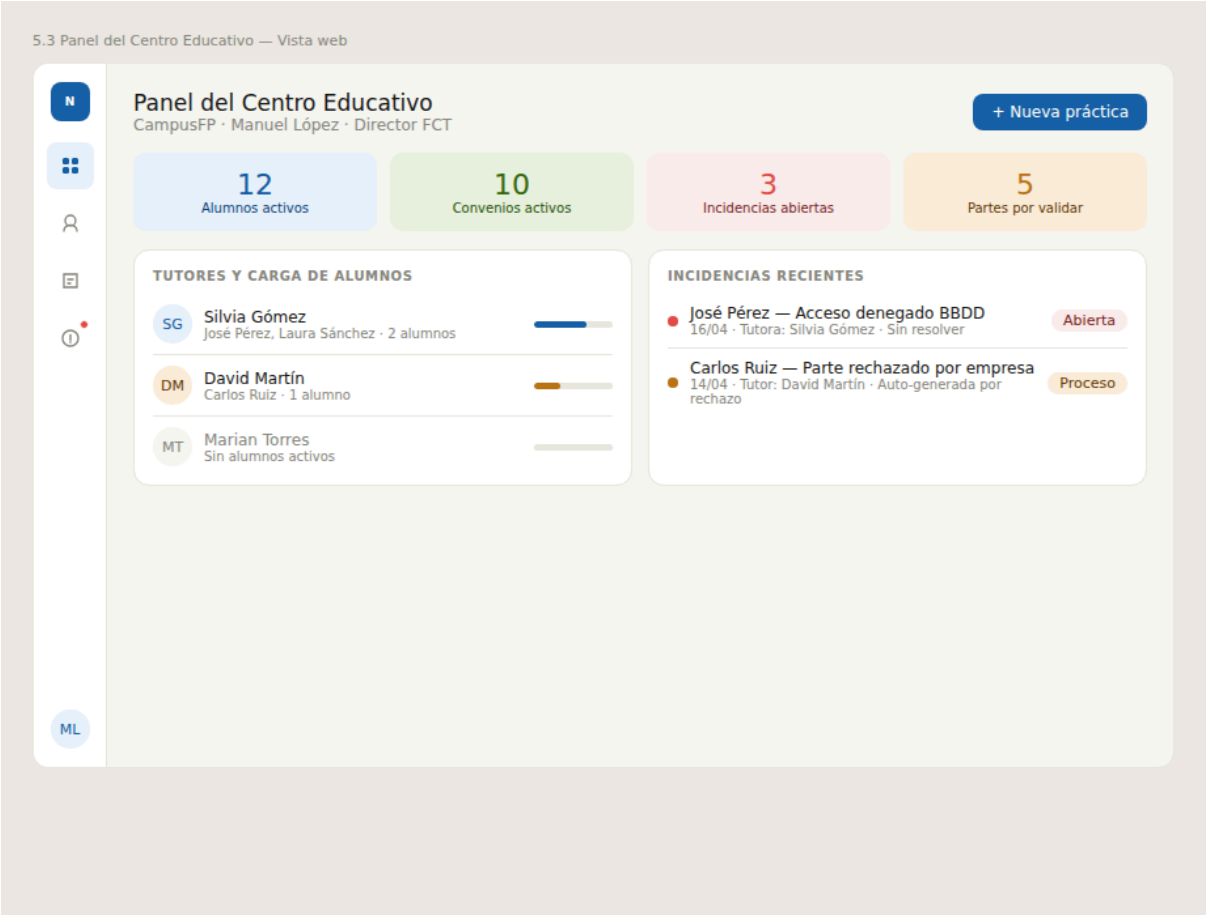
- **Contador de Horas**: Una barra de progreso que te dice de un vistazo cuánto te queda para terminar.
- **Seguimiento Semanal**: Un espacio para escribir qué has hecho cada día. Esto elimina los informes pesados al final del mes.
- **Botón de Incidencias**: Si hay un marrón (tareas que no tocan, problemas de horario), se reporta aquí oficialmente.
- **Chat**: Hablar con el tutor de forma directa y profesional.

5.2. Panel del Tutor



- **Lista de Alumnos**: El tutor puede saltar de un alumno a otro para ver cómo van.
- **Alertas**: Indicadores visuales (como puntos rojos) que avisan si alguien ha reportado una incidencia o no ha rellenado el diario.

5.3. Panel del Centro Educativo



- **Gestión de Tutores**: Ver cuánta carga de trabajo tiene cada profesor.
- **Estadísticas**: Un resumen de cuántos alumnos hay en prácticas y qué convenios están activos.

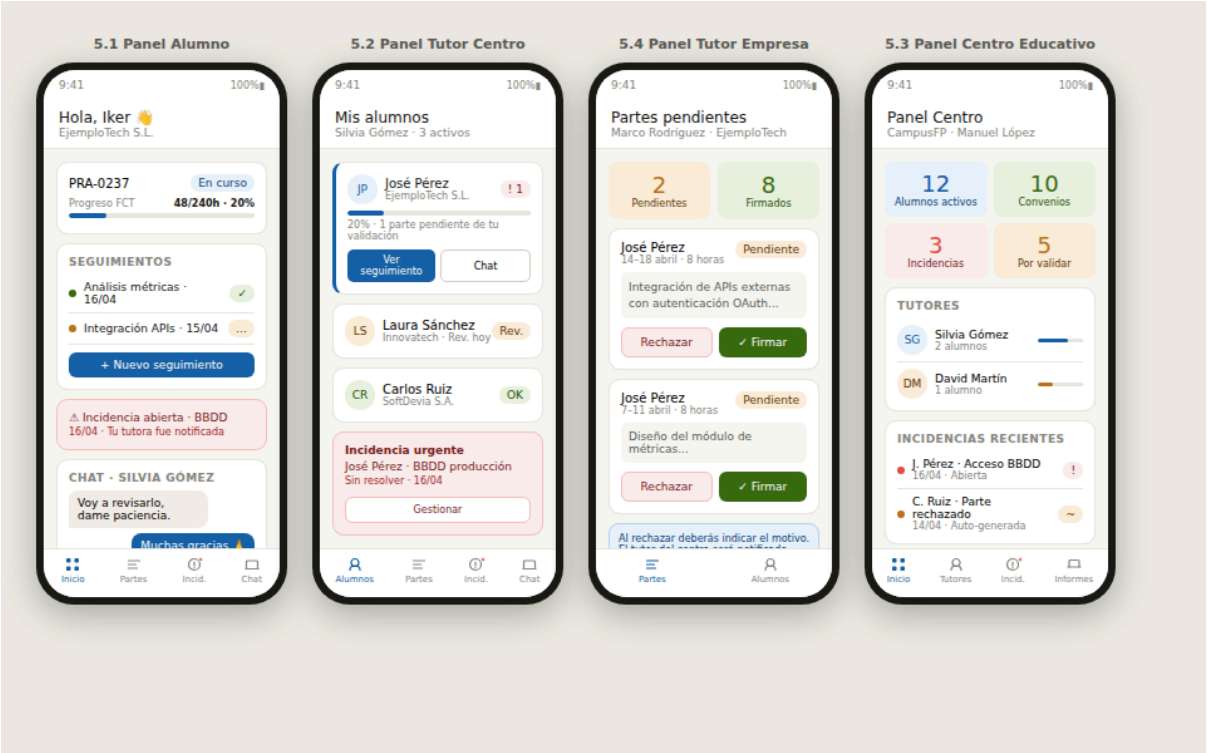
5.4. Panel del Tutor de Empresa

Al añadir la doble validación, necesité diseñar una pantalla específica para el tutor de empresa. Es la más sencilla de todas a propósito: su único trabajo es ver los partes que están esperando su firma y aprobarlos o rechazarlos.

He decidido mantenerla así de simple porque el tutor de empresa no necesita ver el historial académico del alumno, ni las incidencias del centro, ni el chat. Si le meto todo eso en una pantalla se vuelve confusa y deja de parecerse al proceso que ya conoce: revisar el parte de la semana y firmarlo.

Cuando rechaza un parte tiene que escribir el motivo obligatoriamente. Así queda registrado y el tutor del centro sabe exactamente qué pasó.

5.5. Los cuatro paneles en versión móvil



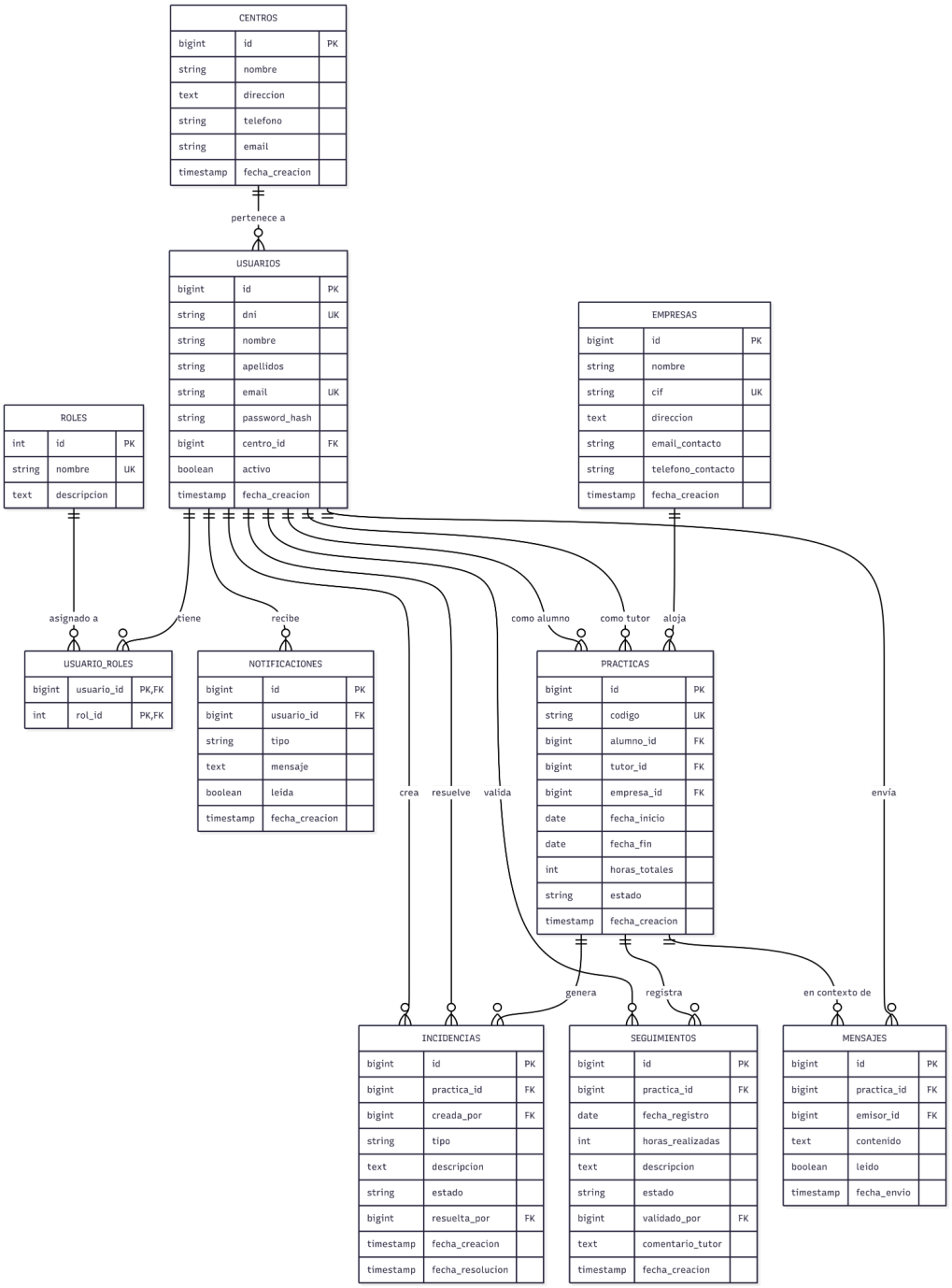
6. Modelo de la Base de Datos

Para que todo esto funcione por detrás, he diseñado un sistema de tablas conectadas:

- **Usuarios:** Donde guardamos quién es alumno, quién es tutor y quién es del centro.
- **Prácticas:** La tabla principal que une al alumno con su empresa y su tutor.
- **Seguimientos e Incidencias:** Registros de lo que se va haciendo y de los problemas que surgen.
- **Chat:** Guardamos todos los mensajes para que haya un historial de lo hablado.

Durante el desarrollo, detecté que un alumno de FP necesita dos responsables: el **Tutor del Centro** y el **Tutor de la Empresa**. He actualizado el modelo para separar ambas figuras en la tabla de prácticas, garantizando un seguimiento académico y laboral independiente..

El resultado final de las tablas interconectadas en PostgreSQL es el siguiente diagrama de Entidad-Relación:



En esta fase se han completado las entidades JPA en el backend, asegurando una sincronización total entre el código y el diagrama Entidad-Relación. Esto incluye la implementación de las clases Java para Incidencias, Mensajes y Notificaciones, que estaban definidas en la base de datos pero pendientes de desarrollar en el servidor.

7. Planificación (Cómo lo voy a hacer)

He dividido el trabajo en 16 semanas para llegar bien a la entrega:

1. **Semanas 1-3:** Analizar todo bien y diseñar las tablas de la base de datos.
2. **Semanas 4-7:** Programar el cerebro (backend) y el sistema para entrar en la web (login).
3. **Semanas 8-12:** Montar las pantallas (frontend), el chat y el sistema de incidencias.
4. **Semanas 13-16:** Probar que todo funcione, corregir fallos y terminar de escribir esta memoria.

Estado Actual (Entrega Hito 1 - 25%): En la primera entrega se completaron las Semanas 1 a 3, con el análisis y diseño de la base de datos mediante Flyway en PostgreSQL, y se avanzó drásticamente en las Semanas 4 a 7. El backend contaba con la estructura base, el sistema de cifrado de contraseñas, la generación de tokens JWT y los endpoints funcionales para el login, el registro y la consulta de entidades maestras.

Estado Actual (Entrega Hito 2 - 50%): Se ha alcanzado el ecuador del proyecto. El backend cuenta ahora con la lógica core completa: CRUD de prácticas con control de estados, sistema de seguimientos con validación por tutores, seguridad real basada en roles con @PreAuthorize en cada endpoint, y una batería de 10 tests de integración que verifican los flujos críticos del sistema. En el frontend Flutter se ha superado la fase de esqueleto inicial: la pantalla de login está conectada a la API en tiempo real, el dashboard muestra los datos reales de la práctica activa del alumno, y el cliente HTTP está configurado con interceptores que inyectan el token JWT automáticamente en cada petición. Durante esta fase se detectó y corrigió un error de diseño en el flujo de validación de seguimientos, separando las responsabilidades del tutor de empresa y el tutor del centro en dos fases diferenciadas, lo que refleja con mayor fidelidad el proceso real de las FCT.

El rediseño del módulo de seguimientos no estaba planificado inicialmente, pero detectarlo a tiempo fue clave. Cambiarlo ahora, antes de tener las pantallas construidas, fue relativamente sencillo. Habría sido mucho más costoso descubrirlo al final.

8. Conclusión y Futuro

Este proyecto demuestra que se puede mejorar mucho la experiencia de las prácticas si dejamos de usar el correo para todo. Al centrarme en el seguimiento, consigo una herramienta que realmente ayuda al alumno y al tutor en su día a día.

En el futuro, la idea es que la aplicación también gestione la firma digital de los convenios de las empresas y que envíe notificaciones al móvil para que nadie se olvide de rellenar sus

horas y el uso de IA para analizar el grado de satisfacción del alumno a través de sus diarios de prácticas. Vamos, encargarse también no solo de la gestión de cuando el alumno ya esta de practicas, si no también encargarse de todo el proceso para conseguir prácticas que la empresa suba cuantas personas y los perfiles que necesita, el alumno preferencias, y que se le da mejor, y el centro se encargue de hacer ese enlace entre empresa y alumno. Pero eso es plan de futuro.